

Advanced Requirement Traceability & Release Readiness

- **Courses:**

- Business Analysis: Writing Effective Acceptance Criteria & Gherkin (Given-When-Then)
- Non-Functional Requirements (NFRs) for Enterprise Applications
- Business Analysis capstone / case-study practice



Summary: Gherkin's Given–When–Then

Gherkin is a plain-English format used in **Behavior-Driven Development (BDD)** to describe system behavior and acceptance criteria in a way that both technical and non-technical stakeholders can understand.

1. **Given – Initial Context**
Defines the starting conditions or system state before any action occurs.
2. **When – Action or Trigger**
Describes the user action or event that triggers the behavior.
3. **Then – Expected Outcome**
Specifies the result that should occur after the action.
4. **And / But – Additional Conditions**
Used to add extra actions, conditions, or validations to improve readability.



Example: User Login

1. Given the user is on the login page
2. When the user enters valid credentials
3. And clicks the login button
4. Then the dashboard should be displayed
5. And a welcome message should appear

Key takeaway: Gherkin's Given–When–Then format creates **clear, testable, and user-focused acceptance criteria**, helping business and technical teams share a common understanding of requirements.



Acceptance Criteria

Acceptance criteria define the conditions that must be met for a user story to be considered complete and successful from the user's perspective.

1. **Define success** – Acceptance criteria describe how the user story should behave and what outcomes the user should experience.
2. **Owned by Product Owner or BA** – The Product Owner or Business Analyst typically creates and documents the acceptance criteria with input from the team.
3. **Include rules and requirements** – They may cover business rules, scenarios, logic, and non-functional requirements such as security, performance, and response times.
4. **Use Given–When–Then format**
 - **Given:** The starting condition or context.
 - **When:** The user action.
 - **Then:** The expected system response.
5. **Focus on the user perspective** – Criteria should describe outcomes users can verify, not technical implementation details.
6. **Support discussion and testing** – They provide enough detail for the team to understand, develop, and test the user story effectively.

Key takeaway: Acceptance criteria clearly define **what success looks like for the user**, helping the team build and test the right solution while ensuring the user story meets expectations.



Non-Functional Requirements (NFRs)

Non-Functional Requirements (NFRs) define **how a system performs**, rather than **what it does**. They describe the quality, performance, security, and operational characteristics of a system.

quality, performance, security, and operational characteristics of a system.

Functional Requirement	Non-Functional Requirement
What the system does	How the system behaves
User can reset password	Password reset completes within 3 seconds

Why NFRs Matter

Even if all functional requirements work correctly, poor NFRs can result in:

-  Slow system performance
-  System crashes under heavy load
-  Security vulnerabilities
-  Poor user experience
-  Difficult maintenance and support

10 Common Types of NFRs

1.  **Security** – Data protection, encryption, authentication, and threat prevention.
2.  **Capacity** – Storage, memory, and processing requirements.
3.  **Compatibility** – Supported devices, hardware, operating systems, and browsers.
4.  **Reliability & Availability** – System uptime, fault tolerance, and stability.
5.  **Maintainability & Manageability** – Ease of updates, fixes, monitoring, and administration.
6.  **Scalability** – Ability to handle increased users, data, or workload.
7.  **Usability** – Ease of use, accessibility, and user experience.
8.  **Performance** – Response time, throughput, and resource utilization.
9.  **Compliance** – Adherence to legal, regulatory, and industry standards.
10.  **Environmental** – Operation under specific physical conditions such as temperature, humidity, or vibration.

Examples of NFRs

-  System response time must be **under 2 seconds**.
-  All customer data must be **encrypted at rest and in transit**.
-  System must support **10,000 concurrent users**.
-  System availability must be **99.9% uptime**.
-  Application must meet **WCAG accessibility standards**.

Key takeaway: NFRs define the **quality attributes and constraints** of a system. They ensure the solution is not only functional but also **secure, reliable, scalable, usable, and performant**, helping meet user, business, and operational expectations.